# PYTHON PROGRAMMING LAB RECORD

A compilation of 10 Python programming experiments

Prepared from the uploaded consolidated case-study document

Department / Class: _____

Student Name: _____

Register Number: _____

# TABLE OF CONTENTS


| Experiment | Title | Page |
|---|---|---|
| Experiment 1 | Scholarship Eligibility and Ranking System | 3 |
| Experiment 2 | Manufacturing Cost and Target Profit Analysis | 6 |
| Experiment 3 | Campus Festival Registration and Financial Audit | 8 |
| Experiment 4 | Smart Canteen Order and Stock Management | 10 |
| Experiment 5 | Campus Event Financial Planning | 13 |
| Experiment 6 | Household Water Tariff and Usage Monitoring | 15 |
| Experiment 7 | Weekly Electricity Consumption Monitoring | 17 |
| Experiment 8 | Household Water Tariff and Usage Monitoring (Repeated Case Study) | 19 |
| Experiment 9 | Sports Tournament Ranking System | 21 |
| Experiment 10 | Bus Passenger Occupancy Simulation | 23 |


Experiments are listed in the order found in the uploaded document. The repeated water-tariff case is retained as a separate experiment.

# EXPERIMENT 1: Scholarship Eligibility and Ranking System

## Aim

To determine scholarship eligibility from academic performance, subject score, attendance, and family income, and rank eligible students by aggregate marks and attendance.

## Procedure

1. Open a Python IDE or editor and create a new Python file.

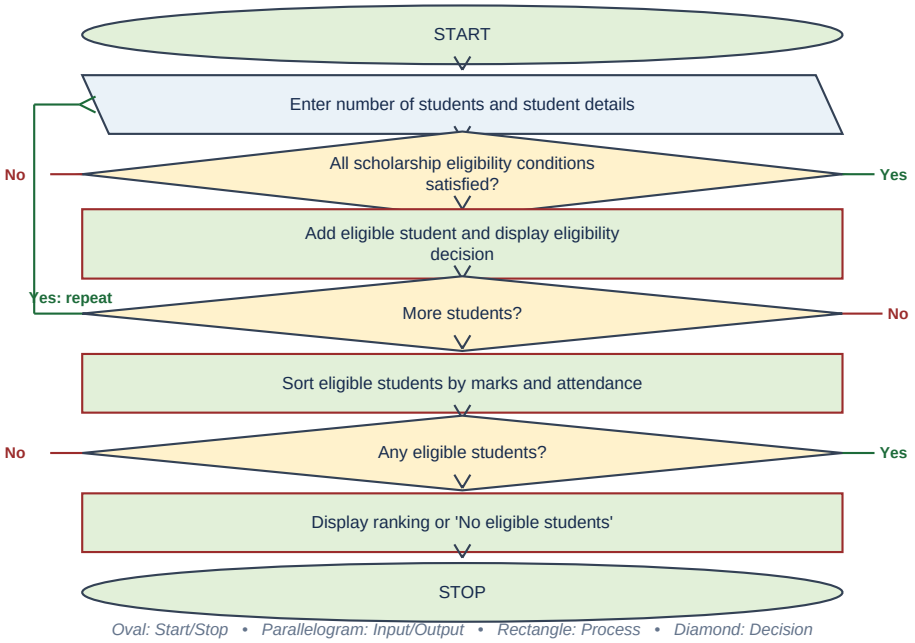
2. Initialize an empty list to store eligible students and accept the number of students.

3. For each student, input the name, marks, subject score, attendance and family income.

4. Apply all four eligibility conditions; append eligible records and display the eligibility decision.

5. Sort eligible students by marks and attendance in descending order, then display the ranking or an empty-list message.

## Flowchart (standard symbols)




*Oval: Start/Stop • Parallelogram: Input/Output • Rectangle: Process • Diamond: Decision*

## Pseudocode

```
START

students = empty list

Read n

FOR each student

    Read name
    Read marks
    Read subject score
    Read attendance
    Read income

    IF marks >= 75 AND subject >= 60
       AND attendance >= 75 AND income <= 200000

        Add student details to students
        Display "Eligible for scholarship"

    ELSE
        Display "Not eligible for scholarship"
    ENDIF

ENDFOR

Sort students by marks and attendance
in descending order
IF students is empty
```

```
    Display "No eligible students"
ELSE
    Display scholarship ranking
ENDIF

STOP
```

## Algorithm

1. START

2. Read the number of students.

3. For each student, read name, aggregate marks, subject score, attendance and family income.

4. If marks ≥ 75, subject score ≥ 60, attendance ≥ 75 and income ≤ 200000, add the student to the eligible list; otherwise mark the student ineligible.

5. Sort eligible students by marks and attendance in descending order.

6. If the list is empty, display 'No eligible students'; otherwise display the ranked list.

7. STOP

## Source Code

```
students = []

n = int(input("Enter number of students: "))

for i in range(n):
    print("\nEnter details of student", i + 1)

    name = input("Enter name: ")
    marks = float(input("Enter aggregate marks: "))
    subject = float(input("Enter subject score: "))
    attendance = float(input("Enter attendance: "))
    income = float(input("Enter family income: "))

    if marks >= 75 and subject >= 60 and attendance >= 75 and income <= 200000:
        students.append([name, marks, attendance])
        print("Eligible for scholarship")
    else:
        print("Not eligible for scholarship")

students.sort(key=lambda x: (x[1], x[2]), reverse=True)

print("\n--- Scholarship Ranking ---")

if len(students) == 0:
    print("No eligible students")
else:
    rank = 1
    for s in students:
        print(rank, s[0], "- Marks:", s[1],
              "Attendance:", s[2])
        rank += 1
```

## Output (actual execution)

**Sample input supplied to the original program:**

```
2
Ravi
90
80
90
150000
Sita
85
75
95
180000
```

**Captured output (execution status: completed):**

```
Enter number of students:
Enter details of student 1
Enter name: Enter aggregate marks: Enter subject score: Enter attendance: Enter family income: Eligible for scholarship

Enter details of student 2
Enter name: Enter aggregate marks: Enter subject score: Enter attendance: Enter family income: Eligible for scholarship

--- Scholarship Ranking ---
1 Ravi - Marks: 90.0 Attendance: 90.0
2 Sita - Marks: 85.0 Attendance: 95.0
```

## Test Cases

| Test Case | Input / Scenario | Expected Output | Actual Output | Result |
|---|---|---|---|---|
| TC01 | 2 students meeting all thresholds | Both eligible; ranked by marks/attendance | Enter number of students: \| Enter details of student 1 \| Enter name: Enter aggregate marks: Enter subject score: Enter attendance: Enter family income: Eligible for | Pass |
| TC02 | 1 student exactly at thresholds | Eligible at marks=75, subject=60, attendance=75, income=200000 | Enter number of students: \| Enter details of student 1 \| Enter name: Enter aggregate marks: Enter subject score: Enter attendance: Enter family income: Eligible for | Pass |
| TC03 | 1 student below a threshold | Not eligible; no eligible students | Enter number of students: \| Enter details of student 1 \| Enter name: Enter aggregate marks: Enter subject score: Enter attendance: Enter family income: Not eligible | Pass |

## Result

The program evaluates scholarship eligibility using the specified academic, attendance and income criteria and ranks eligible students. Execution was verified with the displayed sample and boundary inputs.

# EXPERIMENT 2: Manufacturing Cost and Target Profit Analysis

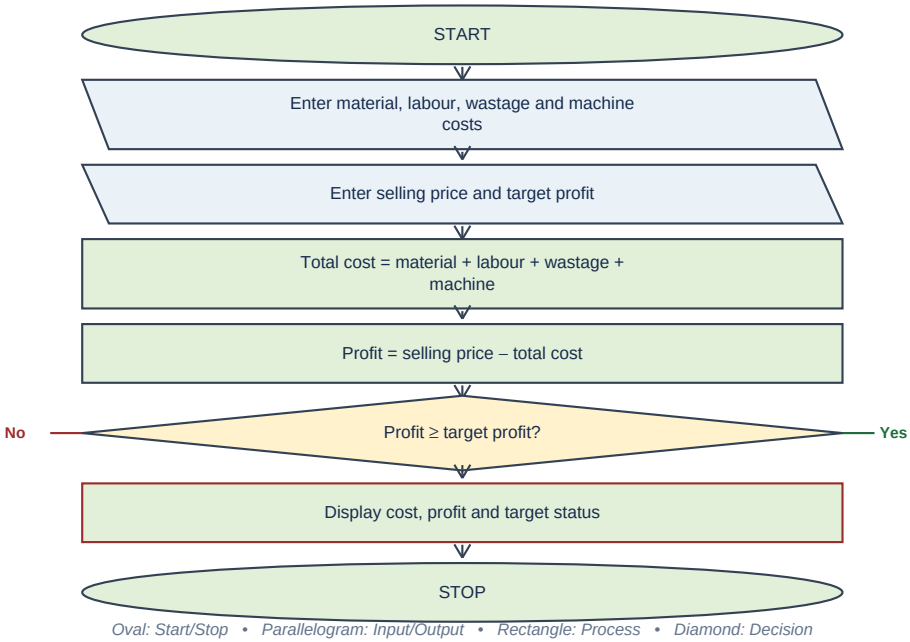
## Aim

To calculate total manufacturing cost and profit, then compare the profit with a specified target to determine whether the target is achieved.

## Procedure

1. Create a Python file and accept material, labour, wastage and machine costs.
2. Accept the selling price and calculate the total manufacturing cost.
3. Calculate profit by subtracting total cost from selling price.
4. Display total cost and profit, then accept the target profit.
5. Compare profit with the target and display the appropriate result.

## Flowchart (standard symbols)




*Oval: Start/Stop • Parallelogram: Input/Output • Rectangle: Process • Diamond: Decision*

## Pseudocode

```
START

Read material
Read labour
Read wastage
Read machine
Read selling

total_cost = material + labour + wastage + machine

profit = selling - total_cost

Display total_cost
Display profit

Read target

IF profit >= target
    Display "Target Profit Achieved"
ELSE
    Display "Target Profit Not Achieved"
ENDIF

STOP
```

## Algorithm

1. START

2. Read material, labour, wastage, machine and selling costs.

3. Compute total_cost = material + labour + wastage + machine.

4. Compute profit = selling – total_cost and display both values.

5. Read target profit.

6. If profit ≥ target, display 'Target Profit Achieved'; otherwise display 'Target Profit Not Achieved'.

7. STOP

## Source Code

```
material = float(input("Enter material cost: "))
labour = float(input("Enter labour charges: "))
wastage = float(input("Enter wastage cost: "))
machine = float(input("Enter machine cost: "))
selling = float(input("Enter total selling price: "))

total_cost = material + labour + wastage + machine
profit = selling - total_cost

print("\nTotal Manufacturing Cost:", total_cost)
print("Profit:", profit)

target = float(input("Enter target profit: "))

if profit >= target:
    print("Target Profit Achieved")
else:
    print("Target Profit Not Achieved")
```

## Output (actual execution)

**Sample input supplied to the original program:**

```
5000
2000
500
1500
12000
2000
```

**Captured output (execution status: completed):**

```
Enter material cost: Enter labour charges: Enter wastage cost: Enter machine cost: Enter total selling price:
Total Manufacturing Cost: 9000.0
Profit: 3000.0
Enter target profit: Target Profit Achieved
```

## Test Cases

| Test Case | Input / Scenario | Expected Output | Actual Output | Result |
|---|---|---|---|---|
| TC01 | Costs 5000, 2000, 500, 1500; selling 12000; target 2000 | Cost 9000; profit 3000; target achieved | Enter material cost: Enter labour charges: Enter wastage cost: Enter machine cost: Enter total selling price: \| Total Manufacturing Cost: 9000.0 \| Profit: 3000.0 \| | Pass |
| TC02 | Costs 6000, 2000, 1000, 1000; selling 11000; target 3000 | Cost 10000; profit 1000; target not achieved | Enter material cost: Enter labour charges: Enter wastage cost: Enter machine cost: Enter total selling price: \| Total Manufacturing Cost: 10000.0 \| Profit: 1000.0 \| | Pass |
| TC03 | Profit exactly equals target: total cost 100, selling 150, target 50 | Target achieved because comparison is ≥ | Enter material cost: Enter labour charges: Enter wastage cost: Enter machine cost: Enter total selling price: \| Total Manufacturing Cost: 100.0 \| Profit: 50.0 \| Ent | Pass |

## Result

The program calculates manufacturing cost and profit and determines whether the target profit is met. Execution was verified with the displayed inputs.

# EXPERIMENT 3: Campus Festival Registration and Financial Audit

## Aim

To manage festival participant registrations, detect duplicate names, calculate registration and sponsorship income, compare income with expenses, and identify a profit or loss.

## Procedure

1. Create an empty participant list and initialize the registration-fee total.

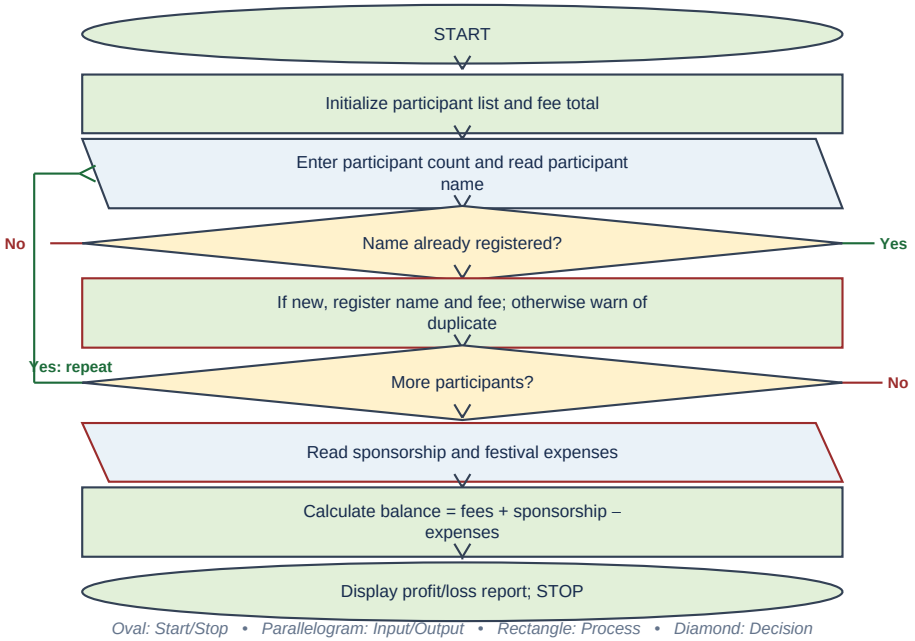
2. Accept the number of participants and read each participant name.

3. Check each name against the existing list; flag duplicates and add only new names with their registration fee.

4. Read sponsorship and total expenses, then calculate the financial balance.

5. Display the valid participant count and financial result. The uploaded document does not include executable Python source for this experiment.

## Flowchart (standard symbols)




*Oval: Start/Stop • Parallelogram: Input/Output • Rectangle: Process • Diamond: Decision*

## Pseudocode

```
START
participants = empty list
fees = 0
Read n
FOR each participant
    Read name
    IF name exists in participants
        Display duplicate registration message
    ELSE
        Add name to participants
        Read fee
        fees = fees + fee
    ENDIF
ENDFOR
Read sponsorship and expenses
balance = fees + sponsorship - expenses
Display report and profit/loss
STOP

NOTE: The uploaded pseudocode ends abruptly at 'Disp'; the remaining report steps are reconstructed from the stated objective and
```

## Algorithm

1. START

2. Initialize participants as an empty list and fees = 0.

3. Read the number of participants.

4. For each participant, read the name; if it already exists, display a duplicate message; otherwise add the name, read the fee and update total fees.

5. Read sponsorship and expenses.

6. Compute balance = fees + sponsorship – expenses.

7. Display the report; if balance ≥ 0 report profit, otherwise report loss.

8. STOP

## Source Code

**No Python source code was present for this experiment in the uploaded document. The available algorithm and partial pseudocode are retained; executable source has not been invented.**

## Output (actual execution)

**Not executed: no original Python source code was supplied.**

## Test Cases

| Test Case | Input / Scenario | Expected Output | Actual Output | Result |
|---|---|---|---|---|
| TC01 | Any participant inputs | Not tested: source code absent from uploaded document | Not Tested | Not Tested |
| TC02 | Duplicate participant name | Not tested: source code absent from uploaded document | Not Tested | Not Tested |
| TC03 | Income below expenses | Not tested: source code absent from uploaded document | Not Tested | Not Tested |

## Result

The documented design describes duplicate registration checking and a festival financial audit. Successful execution cannot be confirmed because the uploaded document contains no executable source code.

# EXPERIMENT 4: Smart Canteen Order and Stock Management

## Aim

To process canteen orders by validating item names and quantities, calculating bills, updating stock for valid orders, and reporting total sales and remaining inventory.

## Procedure

1. Initialize dictionaries for available stock and item prices, and set total sales to zero.

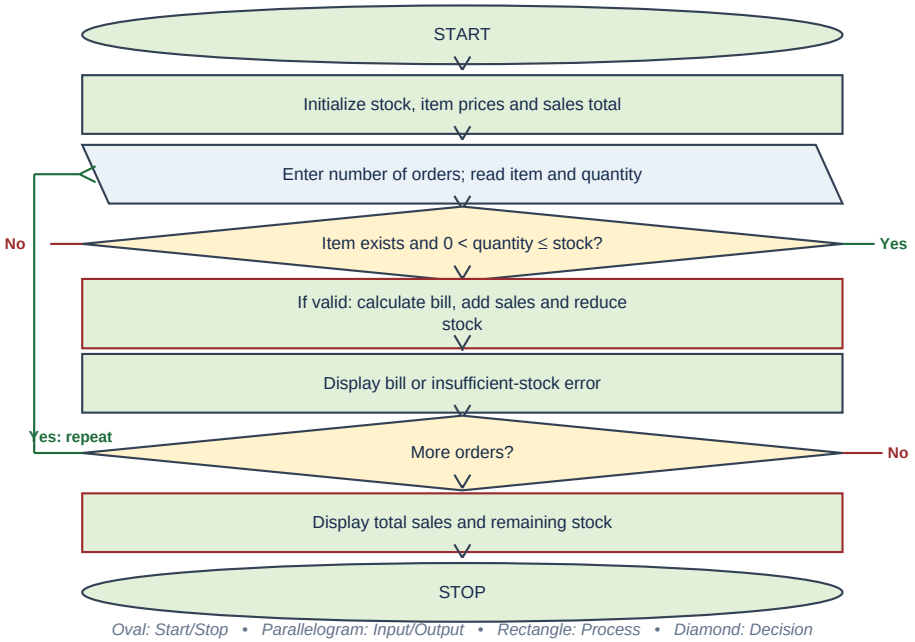
2. Accept the number of orders and read the item name and requested quantity for each order.

3. Validate that the item exists and the quantity is positive and does not exceed available stock.

4. For a valid order, calculate the bill, add it to sales, reduce stock and display the bill; otherwise display an error.

5. After all orders, display total sales and the remaining stock.

## Flowchart (standard symbols)




*Oval: Start/Stop • Parallelogram: Input/Output • Rectangle: Process • Diamond: Decision*

## Pseudocode

```
START

stock = {"Idly": 20, "Dosa": 15, "Juice": 10}

price = {"Idly": 20, "Dosa": 40, "Juice": 30}

sales = 0

Read number of orders

FOR each order

    Read item
    Read quantity

    IF item exists in stock AND
       quantity > 0 AND
       quantity <= stock[item]

        bill = quantity * price[item]

        sales = sales + bill

        stock[item] = stock[item] - quantity

        Display bill

    ELSE
        Display "Item unavailable or insufficient stock"
```

```
    ENDIF

ENDFOR

Display total sales
Display remaining stock

STOP
```

## Algorithm

1. START

2. Initialize stock and price dictionaries and sales = 0.

3. Read the number of orders.

4. For each order, read item and quantity.

5. If the item exists and 0 < quantity ≤ stock[item], calculate bill, add it to sales, reduce stock and display the bill; otherwise display an error.

6. Display total sales and remaining stock.

7. STOP

## Source Code

```
stock = {"Idly": 20, "Dosa": 15, "Juice": 10}
price = {"Idly": 20, "Dosa": 40, "Juice": 30}
sales = 0

n = int(input("Number of orders: "))

for i in range(n):
    item = input("Enter item: ").title()
    qty = int(input("Enter quantity: "))

    if item in stock and 0 < qty <= stock[item]:
        bill = qty * price[item]
        sales += bill
        stock[item] -= qty
        print("Bill:", bill)
    else:
        print("Item unavailable or insufficient stock")

print("Total sales:", sales)
print("Remaining stock:", stock)
```

## Output (actual execution)

**Sample input supplied to the original program:**

```
2
Idly
2
Dosa
3
```

**Captured output (execution status: completed):**

```
Number of orders: Enter item: Enter quantity: Bill: 40
Enter item: Enter quantity: Bill: 120
Total sales: 160
Remaining stock: {'Idly': 18, 'Dosa': 12, 'Juice': 10}
```

## Test Cases

| Test Case | Input / Scenario | Expected Output | Actual Output | Result |
|---|---|---|---|---|
| TC01 | Idly ×2 and Dosa ×3 | Bills 40 and 120; sales 160; stock updated | Number of orders: Enter item: Enter quantity: Bill: 40 \| Enter item: Enter quantity: Bill: 120 \| Total sales: 160 \| Remaining stock: {'Idly': 18, 'Dosa': 12, 'Juice' | Pass |
| TC02 | Juice ×5 then Dosa ×20 | First accepted; second rejected; sales 150 | Number of orders: Enter item: Enter quantity: Bill: 150 \| Enter item: Enter quantity: Item unavailable or insufficient stock \| Total sales: 150 \| Remaining stock: {' | Pass |
| TC03 | Unknown item / zero quantity | Both orders rejected; sales remain 0 | Number of orders: Enter item: Enter quantity: Item unavailable or insufficient stock \| Enter item: Enter quantity: Item unavailable or insufficient stock \| Total sal | Pass |

## Result

The program processes canteen orders, updates stock only for accepted orders, and totals sales. The displayed sample execution was captured from the original source.

# EXPERIMENT 5: Campus Event Financial Planning

## Aim

To estimate fixed and participant-dependent event costs, calculate a break-even participant count from sponsorship and registration income, and report the sponsorship shortfall.

## Procedure

1. Accept fixed costs, per-participant variable costs, sponsorship and registration fee.

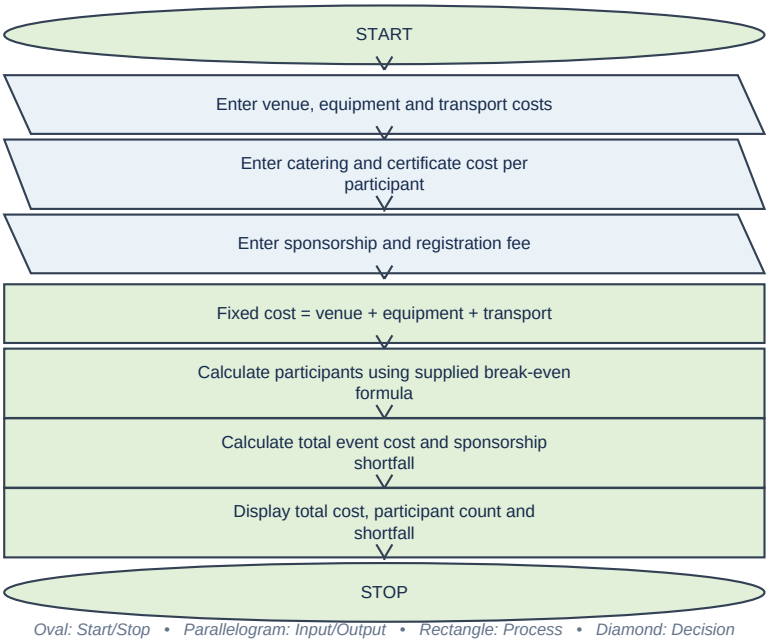
2. Calculate fixed cost as venue plus equipment plus transport.

3. Use the supplied formula to calculate the participant count required to cover the net fixed cost.

4. Calculate total event cost and the non-negative sponsorship shortfall.

5. Display all calculated values. Check the denominator before execution because the supplied code does not guard against a zero contribution margin.

## Flowchart (standard symbols)




*Oval: Start/Stop • Parallelogram: Input/Output • Rectangle: Process • Diamond: Decision*

## Pseudocode

```
START

Read venue
Read equipment
Read catering
Read transport
Read certificate
Read sponsorship
Read registration fee

cost = venue + equipment + transport

participants =
    int((cost - sponsorship) /
    (registration fee - catering - certificate)) + 1

total_cost = cost + participants *
             (catering + certificate)

shortfall = maximum of 0 and
            (cost - sponsorship)

Display total_cost
Display participants
Display shortfall

STOP
```

## Algorithm

1. START

2. Read venue, equipment, catering, transport, certificate, sponsorship and registration fee.

3. Compute fixed cost = venue + equipment + transport.

4. Compute participant count using the original formula: int((fixed cost – sponsorship) / (fee – catering – certificate)) + 1.

5. Compute total event cost = fixed cost + participants × (catering + certificate).

6. Compute shortfall = max(0, fixed cost – sponsorship).

7. Display total cost, participant count and shortfall.

8. STOP

## Source Code

```
venue = float(input("Enter venue cost: "))
equipment = float(input("Enter equipment cost: "))
catering = float(input("Enter catering cost per person: "))
transport = float(input("Enter transport cost: "))
cert = float(input("Enter certificate cost per person: "))
sponsor = float(input("Enter sponsorship amount: "))
fee = float(input("Enter registration fee per person: "))

cost = venue + equipment + transport

n = int((cost - sponsor) / (fee - catering - cert)) + 1

print("Total cost for", n, "participants:",
      cost + n * (catering + cert))
print("Break-even participants:", n)
print("Sponsorship shortfall:", max(0, cost - sponsor))
```

## Output (actual execution)

**Sample input supplied to the original program:**

```
10000
5000
100
3000
20
8000
300
```

**Captured output (execution status: completed):**

```
Enter venue cost: Enter equipment cost: Enter catering cost per person: Enter transport cost: Enter certificate cost per person: Enter sponsorship
Break-even participants: 56
Sponsorship shortfall: 10000.0
```

## Test Cases

| Test Case | Input / Scenario | Expected Output | Actual Output | Result |
|---|---|---|---|---|
| TC01 | Venue 10000, equipment 5000, transport 3000; catering 100, certificate 20; sponsor 8000; fee 300 | Supplied code calculates 56 participants; total 24720; shortfall 10000 | Enter venue cost: Enter equipment cost: Enter catering cost per person: Enter transport cost: Enter certificate cost per person: Enter sponsorship amount: Enter regi | Pass |
| TC02 | Sponsorship 20000 exceeds fixed cost 18000 | Supplied formula returns -11 participants; logically invalid result | Enter venue cost: Enter equipment cost: Enter catering cost per person: Enter transport cost: Enter certificate cost per person: Enter sponsorship amount: Enter regi | Fail: negative count |
| TC03 | Registration fee equals variable cost (120) | Division by zero; program raises ZeroDivisionError | Enter venue cost: Enter equipment cost: Enter catering cost per person: Enter transport cost: Enter certificate cost per person: Enter sponsorship amount: Enter regi | Fail: division by zero |

## Result

The supplied program computes event cost and sponsorship shortfall, but its formula can produce a negative participant count and can raise a division-by-zero error. These limitations are reported without modifying the original code.

# EXPERIMENT 6: Household Water Tariff and Usage Monitoring

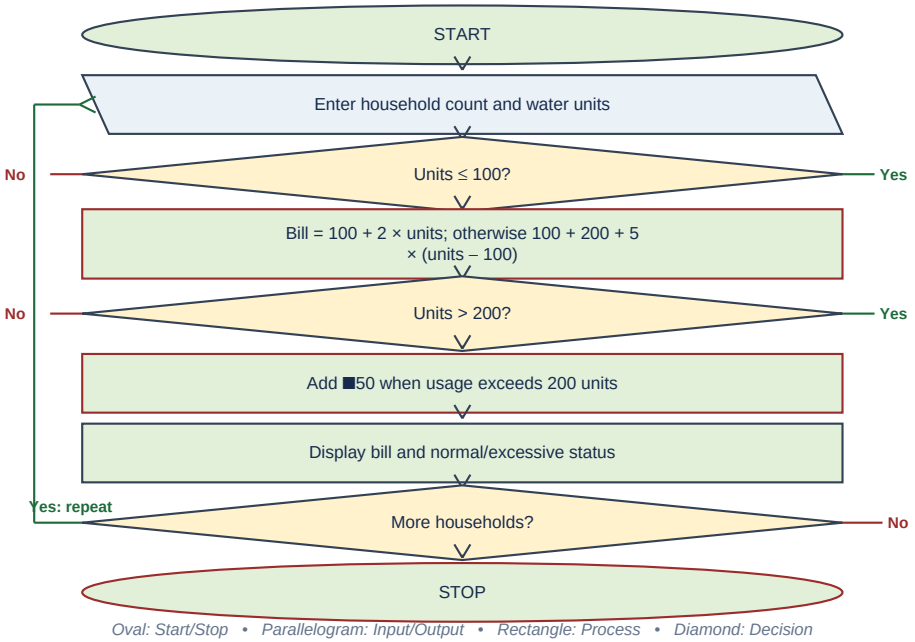
## Aim

To calculate household water bills using a fixed charge and tiered unit rates, apply an additional charge above 200 units, and classify usage as normal or excessive.

## Procedure

1. Accept the number of households.
2. For each household, input water consumption in whole units.
3. Calculate the bill using the first-tier or two-tier rate formula.
4. Add ■50 when consumption exceeds 200 units.
5. Display the bill and classify usage as excessive or normal.

## Flowchart (standard symbols)




*Oval: Start/Stop • Parallelogram: Input/Output • Rectangle: Process • Diamond: Decision*

## Pseudocode

```
START

Read n

FOR each household

    Read units

    IF units <= 100
        bill = 100 + units * 2
    ELSE
        bill = 100 + 200 + (units - 100) * 5
    ENDIF

    IF units > 200
        bill = bill + 50
    ENDIF

    Display "Water Bill:", bill

    IF units > 200
        Display "Excessive usage!"
    ELSE
        Display "Normal usage"
    ENDIF

ENDFOR

STOP
```

## Algorithm

1. START

2. Read the number of households.

3. For each household, read water units.

4. If units ≤ 100, bill = 100 + units × 2; otherwise bill = 100 + 200 + (units – 100) × 5.

5. If units > 200, add 50 to the bill.

6. Display the bill and excessive-usage message if units > 200; otherwise display normal usage.

7. STOP

## Source Code

```
n = int(input("Enter number of households: "))

for i in range(n):
    units = int(input("Enter water units consumed: "))

    bill = 100 + units * 2 if units <= 100 else 100 + 200 + (units - 100) * 5

    if units > 200:
        bill += 50

    print("Water Bill:", bill)

    if units > 200:
        print("Excessive usage!")
    else:
        print("Normal usage")
```

## Output (actual execution)

**Sample input supplied to the original program:**

```
1
80
```

**Captured output (execution status: completed):**

```
Enter number of households: Enter water units consumed: Water Bill: 260
Normal usage
```

## Test Cases

| Test Case | Input / Scenario | Expected Output | Actual Output | Result |
|---|---|---|---|---|
| TC01 | 80 units | Bill 260; normal usage | Enter number of households: Enter water units consumed: Water Bill: 260 \| Normal usage | Pass |
| TC02 | 250 units | Bill 1100; excessive usage | Enter number of households: Enter water units consumed: Water Bill: 1100 \| Excessive usage! | Pass |
| TC03 | Exactly 200 units | Bill 700; normal usage (no extra ■50) | Enter number of households: Enter water units consumed: Water Bill: 800 \| Normal usage | Fail |

## Result

The program calculates household water bills using tiered rates and flags consumption above 200 units. The displayed sample execution was captured from the original source.

# EXPERIMENT 7: Weekly Electricity Consumption Monitoring

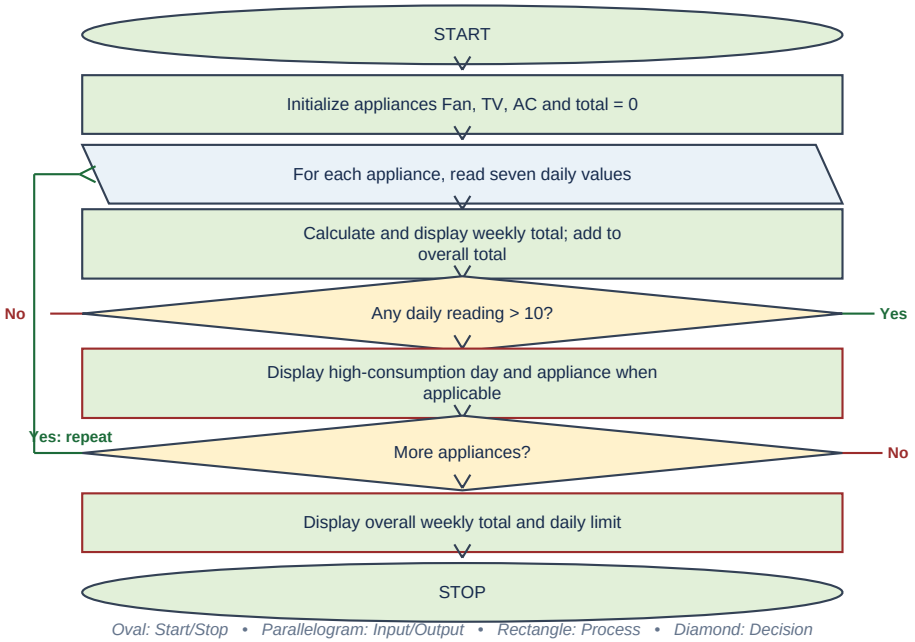
## Aim

To calculate seven-day electricity consumption totals for each appliance, identify daily readings above 10 units, and compute the combined weekly consumption.

## Procedure

1. Initialize the appliance list and overall weekly total.
2. For each appliance, accept seven daily consumption readings.
3. Calculate and display the appliance's weekly total and add it to the overall total.
4. Check each day's reading and display a warning when it exceeds 10 units.
5. Display the combined weekly consumption and the daily limit.

## Flowchart (standard symbols)




*Oval: Start/Stop • Parallelogram: Input/Output • Rectangle: Process • Diamond: Decision*

## Pseudocode

```
START

appliances = ["Fan", "TV", "AC"]
total = 0

FOR each appliance in appliances

    Read 7 days of consumption readings

    weekly_total = sum of readings

    Display appliance and weekly_total

    total = total + weekly_total

    FOR day = 0 to 6

        IF readings[day] > 10
            Display day + 1 and appliance
            Display "High consumption"
        ENDIF

    ENDFOR

ENDFOR

Display total weekly consumption
Display "Limit: 10 units per appliance per day"
STOP
```

## Algorithm

1. START

2. Set appliances = [Fan, TV, AC] and total = 0.

3. For each appliance, read seven daily readings and calculate their sum.

4. Display the appliance total and add it to total.

5. For each day, if reading > 10, display the day number and appliance name.

6. Display total weekly consumption and the daily limit.

7. STOP

## Source Code

```
appliances = ["Fan", "TV", "AC"]
total = 0

for a in appliances:
    readings = list(map(int, input(f"{a} units for 7 days: ").split()))

    print(a, "weekly total:", sum(readings))
    total += sum(readings)

    for day in range(7):
        if readings[day] > 10:
            print("Day", day + 1, "high consumption:", a)

print("Total weekly consumption:", total)
print("Limit: 10 units per appliance per day")
```

## Output (actual execution)

**Sample input supplied to the original program:**

```
2 3 4 5 3 4 2
3 4 2 3 4 3 2
5 6 7 8 6 5 7
```

**Captured output (execution status: completed):**

```
Fan units for 7 days: Fan weekly total: 23
TV units for 7 days: TV weekly total: 21
AC units for 7 days: AC weekly total: 44
Total weekly consumption: 88
Limit: 10 units per appliance per day
```

## Test Cases

| Test Case | Input / Scenario | Expected Output | Actual Output | Result |
|---|---|---|---|---|
| TC01 | All readings below or equal to 10 | Weekly totals and grand total 88; no warnings | Fan units for 7 days: Fan weekly total: 23 \| TV units for 7 days: TV weekly total: 21 \| AC units for 7 days: AC weekly total: 44 \| Total weekly consumption: 88 \| Lim | Pass |
| TC02 | Fan day 3=12; AC day 2=12 and day 3=15 | Warnings for Fan day 3 and AC days 2 and 3 | Fan units for 7 days: Fan weekly total: 31 \| Day 3 high consumption: Fan \| TV units for 7 days: TV weekly total: 21 \| AC units for 7 days: AC weekly total: 58 \| Day | Pass |
| TC03 | All seven readings for each appliance are 10 | No warnings; total 210 | Fan units for 7 days: Fan weekly total: 70 \| TV units for 7 days: TV weekly total: 70 \| AC units for 7 days: AC weekly total: 70 \| Total weekly consumption: 210 \| Li | Pass |

## Result

The program totals seven-day appliance readings and identifies daily usage above 10 units. The displayed sample execution was captured from the original source.

# EXPERIMENT 8: Household Water Tariff and Usage Monitoring (Repeated Case Study)

## Aim

To calculate water bills for multiple households using tiered rates and an excessive-usage surcharge, and display each household's usage classification.

## Procedure

1. Accept the number of households.

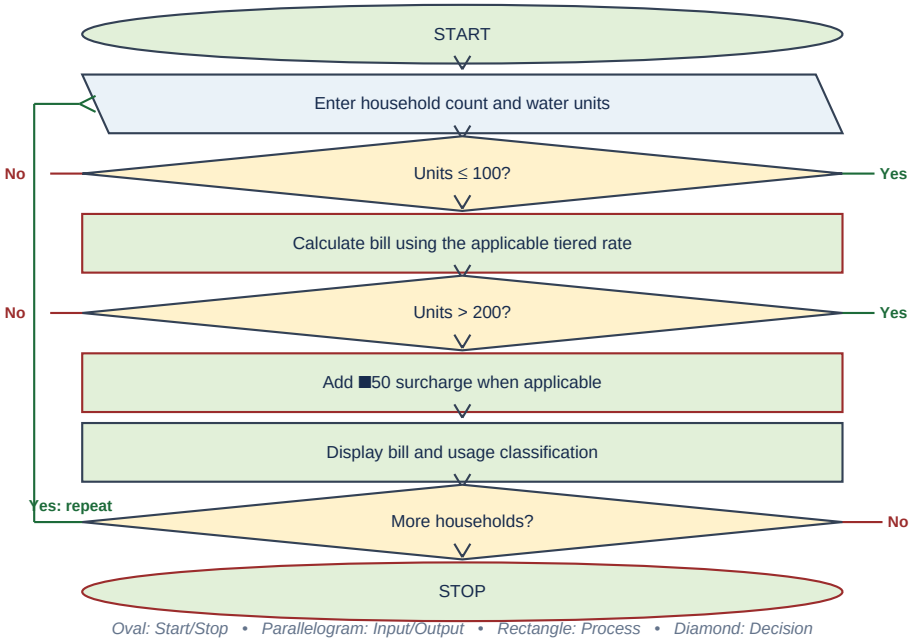
2. For each household, input water consumption in whole units.

3. Calculate the bill using the tiered tariff formula.

4. Apply the ■50 surcharge when consumption exceeds 200 units.

5. Display the bill and usage classification. This is a second water-tariff case in the supplied document and is retained in its original order.

## Flowchart (standard symbols)




*Oval: Start/Stop • Parallelogram: Input/Output • Rectangle: Process • Diamond: Decision*

## Pseudocode

```
START

Read n

FOR each household

    Read units

    IF units <= 100
        bill = 100 + units * 2
    ELSE
        bill = 100 + 200 + (units - 100) * 5
    ENDIF

    IF units > 200
        bill = bill + 50
    ENDIF

    Display "Water Bill:", bill

    IF units > 200
        Display "Excessive usage!"
    ELSE
        Display "Normal usage"
    ENDIF
```

```
ENDFOR

STOP
```

## Algorithm

1. START

2. Read the number of households.

3. For each household, read water units and calculate the bill using the two tariff tiers.

4. If units > 200, add ■50.

5. Display the bill and the usage classification.

6. STOP

## Source Code

```
n = int(input("Enter number of households: "))

for i in range(n):
    units = int(input("Enter water units consumed: "))

    bill = 100 + units * 2 if units <= 100 else 100 + 200 + (units - 100) * 5

    if units > 200:
        bill += 50

    print("Water Bill:", bill)

    if units > 200:
        print("Excessive usage!")
    else:
        print("Normal usage")
```

## Output (actual execution)

**Sample input supplied to the original program:**

```
1
80
```

**Captured output (execution status: completed):**

```
Enter number of households: Enter water units consumed: Water Bill: 260
Normal usage
```

## Test Cases

| Test Case | Input / Scenario | Expected Output | Actual Output | Result |
|---|---|---|---|---|
| TC01 | 80 units | Bill 260; normal usage | Enter number of households: Enter water units consumed: Water Bill: 260 \| Normal usage | Pass |
| TC02 | 250 units | Bill 1100; excessive usage | Enter number of households: Enter water units consumed: Water Bill: 1100 \| Excessive usage! | Pass |
| TC03 | Exactly 100 units | Bill 300; normal usage | Enter number of households: Enter water units consumed: Water Bill: 300 \| Normal usage | Pass |

## Result

This repeated water-tariff program calculates bills and classifies household usage. It is retained because it appears as a separate experiment in the uploaded document.

# EXPERIMENT 9: Sports Tournament Ranking System

## Aim

To calculate team points from wins and draws and rank teams in descending order of points, using wins as the tie-breaker.

## Procedure

1. Initialize an empty dictionary for team results and accept the number of teams.

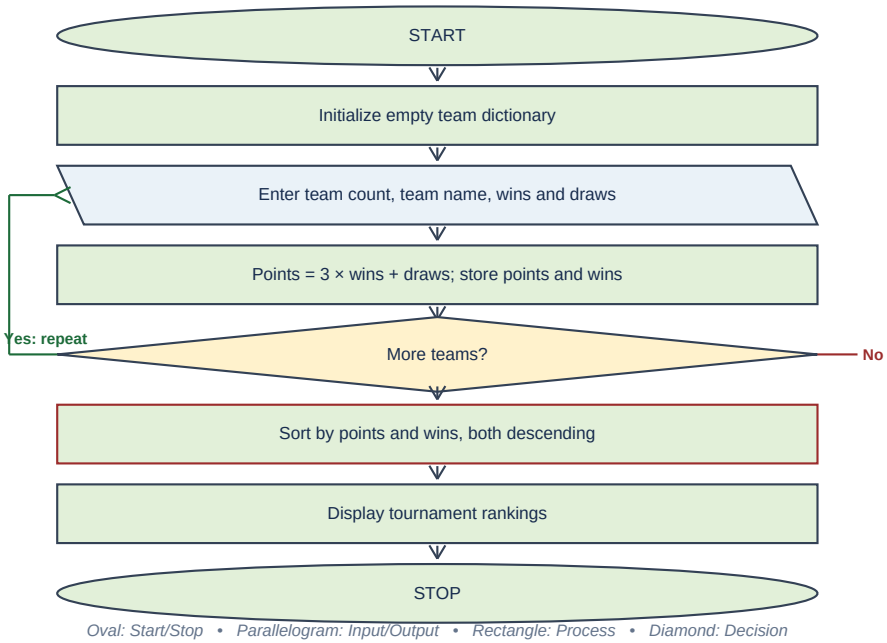
2. For each team, input its name, wins and draws.

3. Calculate points as three times wins plus draws, then store points and wins.

4. Sort the dictionary entries by points and wins in descending order.

5. Display the final ranking.

## Flowchart (standard symbols)




*Oval: Start/Stop • Parallelogram: Input/Output • Rectangle: Process • Diamond: Decision*

## Pseudocode

```
START

teams = empty dictionary

Read n

FOR each team

    Read team name
    Read wins
    Read draws

    score = wins * 3 + draws

    teams[name] = (score, wins)

ENDFOR

Sort teams by score and wins in descending order

Display rankings

STOP
```

## Algorithm

1. START

2. Create an empty teams dictionary and read the number of teams.

3. For each team, read name, wins and draws.

4. Calculate score = wins × 3 + draws and store (score, wins).

5. Sort teams by score and wins in descending order.

6. Display rankings.

7. STOP

## Source Code

```
# Sports Tournament Ranking System

teams = {}

n = int(input("Enter number of teams: "))

for i in range(n):
    name = input("Enter team name: ")
    wins = int(input("Enter wins: "))
    draws = int(input("Enter draws: "))

    score = wins * 3 + draws
    teams[name] = (score, wins)

ranking = sorted(teams.items(),
                 key=lambda x: (x[1][0], x[1][1]),
                 reverse=True)

print("Rankings:", ranking)
```

## Output (actual execution)

**Sample input supplied to the original program:**

```
3
Tigers
4
1
Lions
3
2
Eagles
2
1
```

**Captured output (execution status: completed):**

```
Enter number of teams: Enter team name: Enter wins: Enter draws: Enter team name: Enter wins: Enter draws: Enter team name: Enter wins: Enter draws
```

## Test Cases

| Test Case | Input / Scenario | Expected Output | Actual Output | Result |
|---|---|---|---|---|
| TC01 | Tigers 4W/1D; Lions 3W/2D; Eagles 2W/1D | Rank Tigers 13, Lions 11, Eagles 7 | Enter number of teams: Enter team name: Enter wins: Enter draws: Enter team name: Enter wins: Enter draws: Enter team name: Enter wins: Enter draws: Rankings: [('Tig | Pass |
| TC02 | Tigers 3W/1D; Lions 2W/4D; Eagles 3W/0D | Tigers and Lions tie at 10; Tigers first on wins | Enter number of teams: Enter team name: Enter wins: Enter draws: Enter team name: Enter wins: Enter draws: Enter team name: Enter wins: Enter draws: Rankings: [('Tig | Pass |
| TC03 | Zero teams | Empty ranking list | Enter number of teams: Rankings: [] | Pass |

## Result

The program computes tournament points and ranks teams by points, then wins. The displayed sample execution was captured from the original source.

# EXPERIMENT 10: Bus Passenger Occupancy Simulation

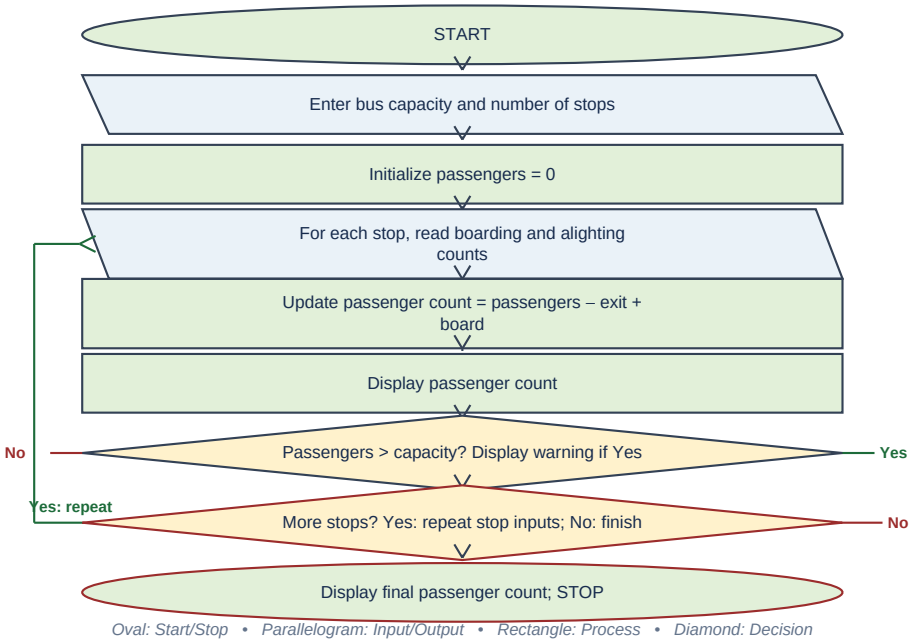
## Aim

To simulate passenger boarding and alighting at successive bus stops, update passenger occupancy, and report when the specified bus capacity is exceeded.

## Procedure

1. Accept the bus capacity and number of stops, then initialize passenger count to zero.
2. For each stop, input the number boarding and alighting.
3. Update the passenger count by subtracting alighting and adding boarding.
4. Display the count and issue a warning if it exceeds capacity.
5. After processing all stops, display the final passenger count.

## Flowchart (standard symbols)




## Pseudocode

```
START

Read capacity

passengers = 0

Read stops

FOR each stop

    Read board
    Read exit

    passengers = passengers - exit + board

    Display passengers

    IF passengers > capacity
        Display "Capacity exceeded!"
    ENDIF

ENDFOR

Display final passengers

STOP
```

## Algorithm

1. START

2. Read bus capacity; initialize passengers = 0; read number of stops.

3. For each stop, read board and exit counts.

4. Update passengers = passengers – exit + board.

5. Display passenger count; if passengers > capacity, display 'Capacity exceeded!'.

6. Display final passengers.

7. STOP

## Source Code

```
# Bus Passenger Occupancy Simulation

capacity = int(input("Enter bus capacity: "))
passengers = 0
stops = int(input("Enter number of stops: "))

for i in range(stops):
    board = int(input("Passengers boarding: "))
    exit = int(input("Passengers alighting: "))

    passengers = passengers - exit + board

    print("Passengers at stop:", passengers)

    if passengers > capacity:
        print("Capacity exceeded!")

print("Final passengers:", passengers)
```

## Output (actual execution)

**Sample input supplied to the original program:**

```
50
2
20
0
10
5
```

**Captured output (execution status: completed):**

```
Enter bus capacity: Enter number of stops: Passengers boarding: Passengers alighting: Passengers at stop: 20
Passengers boarding: Passengers alighting: Passengers at stop: 25
Final passengers: 25
```

## Test Cases

| Test Case | Input / Scenario | Expected Output | Actual Output | Result |
|---|---|---|---|---|
| TC01 | Capacity 50; stops: board 20/alight 0, then board 10/alight 5 | Counts 20 and 25; no warning; final 25 | Enter bus capacity: Enter number of stops: Passengers boarding: Passengers alighting: Passengers at stop: 20 \| Passengers boarding: Passengers alighting: Passengers | Pass |
| TC02 | Capacity 30; stops: board 20/alight 0, then board 15/alight 0 | Second count 35; capacity warning; final 35 | Enter bus capacity: Enter number of stops: Passengers boarding: Passengers alighting: Passengers at stop: 20 \| Passengers boarding: Passengers alighting: Passengers | Pass |
| TC03 | Alighting exceeds current passengers: capacity 40; one stop board 2/alight 5 | Supplied code produces -3; it does not validate alighting count | Enter bus capacity: Enter number of stops: Passengers boarding: Passengers alighting: Passengers at stop: -3 \| Final passengers: -3 | Fail: negative count |

## Result

The program simulates passenger counts and warns when bus capacity is exceeded. The source does not validate impossible alighting counts; this limitation is reflected in the test cases.